Thetawave.ai

💻 物件導向設計 – 封裝與剪刀石頭布遊戲實作

Generated by Thetawave.ai

1.21 OOP 技法四_抽象類別.mp4

🎮 抽象類別與多形在剪刀石頭布遊戲中的應用

> 「抽象類別（Abstract Class）與多形（Polymorphism）是物件導向程式設計（OOP）中實作不同類別間共同行為及差異化行為的關鍵機制。」

抽象類別與多形的核心概念

概念	定義與說明
抽象類別	一種不能直接實體化的類別，用來定義共通屬性與抽象行為（方法），子類別必須實作這些抽象方法。
多形	同一個介面（方法名稱）可對不同類別執行不同的行為，透過覆寫（Override）抽象方法來實現。
抽象方法	抽象類別中未定義具體實作的行為，強制子類別必須實作該方法。

📐 遊戲架構設計與類別模型

遊戲需求重點

- 遊戲為指令行界面（Command Line Interface）互動的剪刀石頭布遊戲。
- 兩位玩家（Player），每位玩家有唯一的編號（Number）。
- 玩家分為兩種：**人類（Human）** 與 **人工智慧（AI）**。
- 人類玩家透過 Camera Interface（類比標準輸入）輸入決策。

- AI 玩家隨機出拳。
- 出拳選項限定為剪刀、石頭、布三種。

Thetawave.ai

類別圖與繼承關係

類別名稱	屬性	主要行為	備註
Player	Number（玩家編號，整數，唯一）	抽象方法：Decide()	抽象類別，強制子類別實作 Decide
HumanPlayer	繼承自 Player	實作 Decide 讀取使用者輸入	透過 Camera Interface 讀入決策
AIPlayer	繼承自 Player	實作 Decide 隨機出拳	使用隨機函式決定出拳
Game	P1、P2（兩位玩家）	Start() 運行遊戲流程	驗證玩家編號唯一，控制遊戲流程

抽象方法 Decide 的多形實作

- Player **類別**：宣告為抽象類別，Decide() 為抽象方法，回傳型態為 Decision（列舉型別）。
- AIPlayer：覆寫 Decide()，隨機選擇剪刀、石頭、布。
- HumanPlayer：覆寫 Decide()，提示使用者輸入數字（1-3），代表剪刀、石頭、布，並進行輸入驗證，若輸入錯誤使用遞迴要求重新輸入。

✂️ 剪刀石頭布遊戲的決策（Decision）列舉（Enum）

列舉值	中文名稱	說明
Paper	布	出布
Scissors	剪刀	出剪刀
Stone	石頭	出石頭

- 每個枚舉有一個 Name 屬性用以顯示中文名稱，覆寫 toString 方法使列舉輸出時顯示中文。

Thetawave.ai

🎲 AI 玩家決策邏輯

- 使用隨機函式從三個選項中選擇一個。
- 以簡化表示為：

$$\text{Decision} = \text{RandomChoice}(\{\text{Paper}, \text{Scissors}, \text{Stone}\})$$

👤 人類玩家決策流程

1. 顯示提示訊息：請輸入 1（剪刀）、2（石頭）、3（布）。
2. 讀取標準輸入（使用 Java 的 Scanner 類別包裝 System.in）。
3. 驗證輸入是否在 1 到 3 之間。
   - 若不合法，顯示錯誤訊息並遞迴呼叫 Decide() 重新輸入。
4. 根據數字對應回傳對應的 Decision。

🏁 遊戲流程（Game.Start() 方法）

1. 驗證兩位玩家的編號是否唯一。
   - 若重複，立即拋出例外錯誤：Player Numbers must be unique。
2. 輪流呼叫兩位玩家的 Decide()，取得出拳決策。
3. 顯示雙方出拳結果（使用 Decision 的中文名稱）。
4. 判斷勝負：
5. 根據判斷結果宣布勝方或平手。

重要實作細節與說明

主題	重點說明
抽象類別實作關鍵字	Java 使用 abstract 關鍵字宣告抽象類別與方法，其他語言實作方式不同，需自行查詢。

建構子傳遞	子類別建構子必須呼叫父類別建構子（Java 用 super(number)），否則會編譯錯誤。
輸入驗證方法	使用遞迴方式重複提示並讀取使用者輸入，確保輸入在允許範圍內。
唯一性驗證	玩家編號唯一性由 Game 類別負責驗證，跨實體（實例）間的約束，避免重複編號。
列舉優化	為列舉加入中文名稱屬性與覆寫 toString()，使輸出更友善易讀。
遊戲邏輯與物件導向分離	剪刀石頭布的勝負判斷邏輯以 Map 或條件判斷實作，與物件導向設計主體分離，保持設計清晰。
Camera Interface	類比標準輸入，用於人類玩家透過命令行輸入決策，透過 Scanner 封裝使用。

代碼示意（Java風格）

```
// 抽象類別 Player
public abstract class Player {
    private final int number;
    public Player(int number) { this.number = number; }
    public int getNumber() { return number; }
    public abstract Decision decide();
}

// AIPlayer
public class AIPlayer extends Player {
    public AIPlayer(int number) { super(number); }
    @Override
    public Decision decide() {
        // 隨機選擇剪刀石頭布
        int choice = new Random().nextInt(3);
        return Decision.values()[choice];
    }
}

// HumanPlayer
public class HumanPlayer extends Player {
    private static Scanner scanner = new Scanner(System.in);

    public HumanPlayer(int number) { super(number); }
    @Override
    public Decision decide() {
        System.out.println("請出拳，1:剪刀，2:石頭，3:布");
```

Thetawave.ai

```
            int input = scanner.nextInt();
            if (input < 1 || input > 3) {
                System.out.println("輸入錯誤，請重新輸入");
                return decide(); // 遞迴重試
            }
            return Decision.values()[input - 1];
        }
    }

    // Decision Enum
    public enum Decision {
        Scissors("剪刀"),
        Stone("石頭"),
        Paper("布");

        private final String name;
        Decision(String name) { this.name = name; }
        @Override
        public String toString() { return name; }
    }
```

按圖施工與設計實務

- **按圖施工**：意指依據設計藍圖作為指引，分割並逐步完成各類別的實作，而非一開始就要完全詳細的設計。
- 設計階段完成大約 60% 的藍圖即可進入編碼，細節可在編碼時調整與補足。
- OOP 實作時，透過抽象類別與多形幫助區分不同類別的共同行為與獨特行為。
- 按圖施工強調「分割即破」，將複雜問題拆解成小部分逐一實作。

多形與抽象類別的整合意義

抽象類別提供了共通的行為定義與資料結構，而多形使得不同類別能依照自身特性實作細節，整體架構中，玩家類別的抽象方法 Decide() 成為連接共通介面與差異化行為的關鍵。此設計使得遊戲邏輯可藉由多形輕鬆擴充，新增其他玩家類

型時只需實作決策方法，遊戲主流程與勝負判斷則無須改動，展現良好的擴充性與維護性。

1.23 OOD 階段二_封裝,解耦,萃取.mp4

🧩 OOD 階段二：封裝、結構與萃取

> **最小知識原則（Least Knowledge Principle）**
> 物件間應該彼此知道越少越好，避免過度耦合，減少修改時的影響範圍。

OOD 階段二的目標與流程

- **目標**：對從物件導向分析（OOA）產出的領域模型初步精煉，提升彈性與維護性。
- **流程**：
  1. **封裝（Encapsulation）**
     降低類別間的可見度，避免不必要的耦合。
  2. **結構（Structure）**
     優化類別間的關聯，降低雙向依賴。
  3. **萃取（Extraction）**
     從複雜結構中抽象出介面或抽象類別，實現依賴反轉。

---

1. 封裝（Encapsulation）

封裝的定義與目的

- 封裝是將類別的所有屬性設為**私有（Private）**，只允許透過特定操作（方法）查詢或修改。
- 目的：**減少類別間的耦合**，避免其他類別直接依賴內部資料結構。

可見度（Visibility）的種類

可見度類型	說明	UML 符號	可見對象
公開（Public）	所有類別皆可存取	+	全部類別
高級（Package）	同一個 package（封包）內的類別可見	~	同一封包內的類別
受保護（Protected）	類別本身及子類別可見	#	自己與子類別
私有（Private）	只有類別本身可見	-	自己類別

> **注意**：UML 中的「類別」可視為抽象分類，不一定等同程式碼中的 class，可能代表模組、套件或微服務。

為何封裝要降低可見度？

- **降低耦合**：可見度越開放，其他類別越容易直接存取內部屬性，造成耦合。

  耦合定義：

  > 若修改 B 類別導致 A 類別必須連帶修改，則 A 耦合 B。

- **耦合的結果**：
  - 修改一個類別時，必須修改多個相關類別，增加維護成本。
  - 例子：若玩家（Player）類別的手牌屬性（Hand Card）是公開的，其他類別直接存取陣列結構，資料結構改變將造成所有依賴程式碼編譯錯誤。

封裝的實務做法

- 將所有屬性設為私有。
- 透過公開方法提供恰當的存取介面，例如提供「取得第 N 張手牌」的查詢方法。
- **範例**：

```
class Player {
  private Card[] handCards;  // 私有屬性

  public Card getHandCard(int index) {  // 透過方法查詢
    return handCards[index];
  }
```

```
}
```

Thetawave.ai

- 修改資料結構（如改用 Link Hash Set）時，外部程式碼不受影響，降低耦合。

可見度的彈性調整

- 例如父類別將屬性設為私有，但允許子類別存取：
  - 將屬性可見度升級為**受保護（Protected）**。
- 設計需有合理論述，明確哪些屬性允許子類別直接使用。

---

2. 解耦（Decoupling）

解耦的定義

- 當系統已有高度耦合時，透過調整結構降低耦合，改善維護性。
- 解耦是封裝後的進一步優化。

解耦的兩個主要實踐思路

實踐思路	說明	範例說明
去除雙向關聯（Bidirectional Association）	儘量改為單向關聯，降低複雜度與耦合度	車（Car）與輪子（Wheel）為單向關聯，輪子不需知道車
不與朋友的朋友說話（Only Talk to Your Friends）	物件只與直接相關的物件互動，避免間接依賴	駕駛（Driver）直接呼叫車（Car）的啟動引擎（Launch Engine）方法，不直接呼叫引擎（Engine）

雙向關聯與單向關聯比較

- **雙向關聯**：A 與 B 互相持有對方引用，建立複雜的同步與管理，耦合較強。
- **單向關聯**：只有 A 知道 B，結構簡單，易維護。

雙向關聯的例外情境

- **狀態雙向同步（State Synchronization）**
  如世界（World）與怪物（Monster）雙向關聯，怪物死亡時需通知世界

移除，雙向關聯利於狀態同步與維護。

Thetawave.ai

---

3. 不與朋友的朋友說話（只和直接物件互動）

問題描述

- 物件間若存在多層存取（多個「點」操作符），代表物件與間接相關的陌生物件互動，增加耦合。

改善方案

- 將間接操作封裝成直接操作，由直接相關物件提供介面。
- **範例**：

不良設計	改良設計
driver.getCar().getEngine().launch()	driver.launchEngine()（由 Car 提供此方法）

- 駕駛（Driver）不需知道引擎（Engine）細節，只與汽車（Car）互動。

---

4. 萃取（Extraction）

- 萃取主要指從複雜類別結構中抽象出**抽象類別**或**介面（Interface）**，達成依賴反轉（Dependency Inversion）。
- 此部分與設計模式密切相關，將在後續章節中深入講解。
- 萃取也是實踐最小知識原則的重要途徑。

封裝與解耦的核心關係

觀點	說明
封裝	預防耦合，從設計初期即降低屬性可見度，避免外部依賴資料結構。

解耦	對已有的複雜耦合結構進行重構，調整類別關聯，簡化依賴關係。
共同目標	降低類別間的耦合度，提升系統彈性與維護性，減少修改帶來的連鎖反應。

Mindwave.ai

重要概念總結

關鍵詞	定義與說明
最小知識原則	物件間應只知道彼此必要的最少資訊，避免不必要的耦合與依賴。
封裝（Encapsulation）	將類別屬性設為私有，僅透過方法提供操作，防止外部直接依賴內部資料結構。
可見度（Visibility）	分為公開、公包、受保護、私有，決定類別屬性與方法的存取範圍，UML以符號標示。
耦合（Coupling）	當一個類別改變會影響另一個類別需同步修改的依賴關係，耦合度越高維護成本越大。
雙向關聯（Bidirectional Association）	兩個類別彼此持有對方的引用，狀態同步複雜且耦合度高，通常應避免。
解耦（Decoupling）	透過重構降低系統中類別間的依賴關係，提升系統的可維護性與彈性。
不與朋友的朋友說話	物件只與直接相關物件互動，避免跨越多層間接互動造成的高耦合。
萃取（Extraction）	抽象出介面或抽象類別，達成依賴反轉，提高設計彈性，與設計模式密切相關。

封裝與解耦的實例說明

情境	問題	解決方案
玩家類別手牌屬性公開	其他類別直接存取陣列結構，資料結構改變導致編譯錯誤	將手牌設為私有，提供「取得手牌」方法，封裝內部資料結構
管理者與員工雙向關聯	雙向持有引用，建立資料同步與維護複雜	改為單向關聯，僅管理者持有員工列表，降低耦合度
駕駛呼叫引擎多層操作	駕駛需先取得汽車，再取得引擎，操作複雜且耦合高	汽車提供啟動引擎方法，駕駛直接呼叫汽車方法，簡化存取路徑

怪物死亡需即時通知世界	怪物與世界狀態需雙向同步	保持雙向關聯，方便狀態即時同步

Mubunove.ai

封裝與最小知識原則的關聯

- 封裝是**實踐最小知識原則的基礎**，透過控制可見度降低對類別的依賴。
- 減少耦合使系統更具彈性，需求變更時影響範圍縮小，維護變得更容易。
- 物件導向設計應自始至終貫徹封裝原則，避免「歡迎耦合」的設計陷阱。

物件導向分析與設計中封裝的抽象層級

抽象層級	類別定義範圍	應用場景
程式碼層級	程式語言中的 class	細部軟體程式架構設計
模組層級	專案中的模組或套件	模組化架構分析
微服務分散式系統	微服務作為類別	分散式系統設計

封裝強化了系統內部的資訊隱藏與邊界管理，減少物件間的直接依賴，使得修改內部實作時不會影響外部使用者。解耦則是在已有系統中重新調整物件間的關聯，去除不必要的雙向依賴及間接互動，提升系統的維護性與擴充性。這兩者共同推動物件導向設計的彈性與穩定性，而萃取則透過抽象層的引入，進一步實現依賴反轉與高內聚低耦合的設計目標。